

Searching for GPU Thread-Block and Tile-Size Configurations in PolyBench-MLIR via Coordinate Descent

A Five-Month Final-Year Project Workplan

Fernando Magno Quintão Pereira

1 Introduction

1.1 PolyBench and Kiriko

PolyBench [5] is a widely used benchmark suite of loop-intensive kernels (linear algebra, stencils, data mining) that has served for over a decade as a standard testbed for polyhedral compilation research. PolyBench-MLIR is a complete, manually validated rewrite of the 30 PolyBench 3.2 kernels in the MLIR Affine dialect [4]. It was produced because no native MLIR version previously existed and automatic C-to-MLIR raising tools (e.g., Polygeist) require substantial manual refinement before the result is usable as a stable benchmark artifact.

Kiriko¹ is the accompanying benchmarking infrastructure: a unified harness that dispatches each kernel through one of several optimizing toolchains (Clang, LLVM Polly, Pluto, MLIR Affine), lowers it to an object file, links it against a common PolyBench driver, and executes it under controlled, repeatable conditions, collecting timing data and hardware performance counters. Kiriko’s design goals – unification, reproducibility, automation, and fairness – make it a natural harness to extend for new experiments rather than build one from scratch.

1.2 Why Lower These Kernels to GPUs?

The current Kiriko evaluation targets CPU toolchains exclusively. Several PolyBench-MLIR kernels (matrix-matrix products, matrix-vector kernels, elementwise stencils) expose data parallelism that MLIR can already recognize through `affine.parallel` loops, making them plausible candidates for GPU execution. Lowering a subset of the suite to the `gpu` dialect would (i) extend Kiriko into a genuinely heterogeneous benchmarking platform, and (ii) let us study a question that is presently unanswered for this benchmark suite: how do tile-size and thread-block configuration choices affect performance once these kernels run on real GPU hardware.

1.3 The Space of Optimization Parameters

Mapping an affine loop nest to the GPU dialect requires choosing, at minimum: (a) a tile size for each loop dimension prior to mapping, which determines how much work is assigned per thread block, and (b) a thread-block shape (its extent along up to three dimensions), which determines occupancy, register pressure, and shared-memory usage [1]. These parameters are not independent: hardware limits (maximum threads per block, shared-memory capacity, register file size) impose

¹Available at <https://github.com/lac-dcc/Kiriko>

joint constraints, and warp granularity (multiples of 32 threads) further restricts the feasible set. The result is a *discrete*, hardware-constrained parameter space rather than a small number of free-standing scalar knobs [2, 3].

1.4 Why Explore This Space, and the Convexity Hypothesis

There is no reliable closed-form model that predicts the best tile size and block shape for a given kernel and problem size: occupancy calculators offer rough guidance, but effects such as memory coalescing, shared-memory bank conflicts, and register spilling introduce sharp discontinuities that analytical models do not capture. Because of this, GPU kernel autotuners (Halide’s autoscheduler, TVM/AutoTVM, Ansor, and, within the MLIR ecosystem itself, Transform-dialect-driven Bayesian search) treat the problem empirically: instantiate a configuration, measure it, and search.

A central open question we want to characterize is what we call the *convexity hypothesis*: does the performance landscape over this discrete parameter space behave, at least approximately, as a well-behaved (unimodal, “convex-like”) surface for most kernels – in which case a cheap local-search strategy such as coordinate descent should reliably find a near-optimal configuration – or is it instead frequently multimodal and cliff-ridden, in which case coordinate descent from a single starting point would often settle for a mediocre local optimum. This “Convexity Hypothesis” has been partially validated by Canesche et al. in the context of the TVM compiler [2]. Establishing how often each behavior occurs, and whether it correlates with kernel characteristics (arithmetic intensity, access pattern, reduction structure), is itself a useful empirical contribution, independent of whether coordinate descent turns out to be a good search strategy in practice.

2 The Goals of the Work

By the end of the project we want to be able to answer the following concrete questions:

- G1. For a chosen subset of embarrassingly-parallel PolyBench-MLIR kernels, what does the feasible discrete space of (tile size, thread-block shape) configurations look like once hardware constraints are applied?
- G2. How does measured performance vary across this feasible space, per kernel, and how “well-behaved” (unimodal vs. multimodal) is that landscape in practice?
- G3. How close to the best configuration found by exhaustive (or near-exhaustive) grid search can a simple coordinate-descent search, started from a fixed baseline configuration, get – and how many measurements does it need to get there?
- G4. Do the answers to (2) and (3) transfer between two GPUs with different architectural generations and resource budgets?

3 The Infrastructure

3.1 Software

- **PolyBench-MLIR**: the 30-kernel benchmark suite in the MLIR Affine dialect, from which we will select the embarrassingly-parallel subset (e.g., `2mm`, `3mm`, `gemm`, `gesummv`, `gemver`, `syrk`, `syr2k`, `atax`, `bicg`, `mvt`, `doitgen`) as the initial scope, deferring kernels with loop-carried dependences (solvers, stencils) to future work.

- **Kiriko**: the existing benchmarking harness, which we will extend with a new GPU backend alongside its current Clang, Polly, Pluto, and MLIR Affine (CPU) backends, reusing its configuration and result-collection conventions where possible.
- **The MLIR/GPU lowering pipeline**: `affine.for` loops proven parallel are converted to `affine.parallel` (via `affine-parallelize`), tiled with a configurable tile size (via the parallel-loop tiling pass), mapped onto GPU blocks and threads (`gpu-map-parallel-loops`), converted to the `gpu` dialect (`convert-parallel-loops-to-gpu`), and finally lowered through the standard `gpu-to-NVVM` path to a CUDA binary. Tile size and block-shape parameters will be exposed as script parameters using the MLIR Transform dialect, so that a driver program can generate and measure many configurations without recompiling the pass pipeline for each one.
- **GPU profiling**: Kiriko currently measures CPU runs with Linux `perf` counters, which cannot see inside GPU execution. We will integrate NVIDIA’s `nsys/ncu` tooling to collect kernel execution time and occupancy-related metrics for each configuration.
- **Search harness**: a lightweight driver, built on top of the above, that instantiates a configuration, compiles and executes it through Kiriko’s GPU backend, records the measurement, and implements both an exhaustive/grid search strategy and a coordinate-descent strategy over the feasible parameter space.

3.2 Hardware

We have access to two GPUs of different architectural generations and resource envelopes, which lets us test whether conclusions about the parameter space and about coordinate descent’s effectiveness transfer across hardware:

- An **NVIDIA RTX 3090** (consumer Ampere-class GPU).
- An **NVIDIA A100** (data-center Ampere-class GPU, larger register file, shared memory, and memory bandwidth).

4 Methodology

The project is scoped for approximately 20 weeks at 6 hours/week (roughly 120 hours), organized around four milestones with a small buffer built in for debugging the GPU lowering path, which is historically the most failure-prone step in this kind of project.

Milestone	Weeks	Deliverable
M1: Kernels on GPU	1–5	End-to-end lowering pipeline working for the chosen embarrassingly-parallel kernel subset, running correctly on the RTX 3090 with output validated against the reference PolyBench/C results (Kiriko’s existing verification infrastructure). Kiriko extended with a minimal GPU backend using a single fixed (tile size, block shape) configuration per kernel.
M2: Parameter space characterization	6–10	Transform-dialect parameterization of tile size and block shape; GPU profiler (<code>nsys/ncu</code>) integrated into Kiriko; feasible discrete configuration space enumerated per kernel under hardware constraints; grid (or near-exhaustive) search executed for 2–3 representative kernels on the RTX 3090, producing the first performance-landscape plots.
M3: Grid search vs. coordinate descent	11–15	Coordinate-descent search implemented, started from a fixed baseline (“origin”) configuration for each kernel; compared against the grid-search ground truth from M2 in terms of (a) distance from the best configuration found and (b) number of measurements required to converge; the full kernel subset covered on the RTX 3090, with a first repetition of the same experiments on the A100 to test cross-GPU transfer.
M4: Consolidation and write-up	16–20	Remaining kernels and any stretch-goal configurations covered; results consolidated across both GPUs; final report written up, characterizing how often the landscape is well-behaved (supporting or refuting the convexity hypothesis) and how reliable coordinate descent is as a cheap default strategy; code and results prepared for public release alongside PolyBench-MLIR and Kiriko.

Throughout, weekly check-ins should track two risks explicitly: (i) lowering failures or miscompilations on specific kernels (mitigated by reusing Kiriko’s existing correctness-checking driver at every step), and (ii) search-harness measurement noise (mitigated by reusing Kiriko’s existing methodology of repeated runs per configuration, already used for its CPU results).

5 Conclusion

We expect this project to produce three concrete outcomes. First, a working GPU backend for Kiriko and a GPU-executable subset of PolyBench-MLIR, extending both artifacts beyond their current CPU-only scope and making them useful to other researchers who want a reproducible, cross-toolchain GPU baseline. Second, an empirical characterization of the tile-size/thread-block performance landscape for a set of well-known kernels on two GPU generations – a modest but concrete data point on the convexity hypothesis, i.e., how often a cheap local-search strategy suffices versus how often it is misled by a multimodal or cliff-ridden landscape. Third, a direct, apples-to-apples comparison between grid search and coordinate descent on this specific parameter space,

including how many measurements each strategy needs to reach a given quality level.

These results should be useful to the MLIR community specifically, by adding a public, GPU-capable extension to PolyBench-MLIR and Kiriko that other researchers can build on for future autotuning or scheduling work using the Transform dialect. More broadly, they contribute to the compiler and autotuning community’s understanding of when simple, cheap search strategies are sufficient for GPU kernel tuning and when more sophisticated search (Bayesian optimization, genetic search, learned cost models) is actually warranted – a question of practical importance given how routinely such tuning appears in production ML and HPC compiler pipelines.

References

- [1] Michael Canesche. [RFC] adding an exploitation phase to metaschedule to improve scheduling. <https://discuss.tvm.apache.org/t/rfc-metaschedule-adding-an-exploitation-phase-to-metaschedule-to-improve-scheduling/17365>, 2024. Experimental results are listed on a report that is publicly available at https://homepages.dcc.ufmg.br/~michaelcanesche/paper/bennu_meta_version.pdf.
- [2] Michael Canesche, Vanderson M. Rosario, Edson Borin, and Fernando Magno Quintão Pereira. The droplet search algorithm for kernel scheduling, feb 2024. Just Accepted.
- [3] Michael Canesche, Gaurav Verma, and Fernando Magno Quintao Pereira. Explore as a storm, exploit as a raindrop: On the benefit of fine-tuning kernel schedulers with coordinate descent, 2024.
- [4] Lucas Victor Costa, Jose Wesley Magalhaes, Michael Canesche, and Fernando Magno Quintao Pereira. An mlir version of the polybench benchmark suite for high-performance computing. Technical report, Cadence, 2026.
- [5] Louis-Noël Pouchet et al. Polybench: The polyhedral benchmark suite, 2012.